Question 3 - Prometheus and Grafana monitoring

Question 3: Set up monitoring using Prometheus and visualize metrics using Grafana dashboards for a target application or system.

All commands below were run on the lab machine and their output is copied verbatim from the recorded logs in `evidence/logs/`. Repository: https://github.com/ksamirdev/ensp461-devops-lab3.

Instrumentation

service/web.py (metrics)

```
REQUESTS = Counter(
    "textstats_http_requests_total", "HTTP requests served", ["method", "route", "status"]
)
LATENCY = Histogram(
    "textstats_http_request_duration_seconds",
    "Request latency",
    ["route"],
    buckets=(0.0005, 0.001, 0.0025, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1),
)
WORDS = Counter("textstats_words_processed_total", "Words analysed by /stats")
```

service/web.py (one hook observes every request)

```
@app.after_request
def observe(response):
    if request.path == "/metrics":
        return response
    # url_rule, not path, so unknown URLs cannot create unbounded label values
    route = request.url_rule.rule if request.url_rule else "unmatched"
    elapsed = time.perf_counter() - g.started
    REQUESTS.labels(request.method, route, str(response.status_code)).inc()
    LATENCY.labels(route).observe(elapsed)
    level = logging.WARNING if response.status_code >= 400 else logging.INFO
    log.log(
        level,
        "method=%s path=%s status=%s duration_ms=%.2f",
        request.method, route, response.status_code, elapsed * 1000,
    )
    return response
```

Routes are labelled by their URL rule, not the raw path. Unknown URLs all share `route="unmatched"`, so random URLs cannot create unlimited metric series (a unit test covers this).

Prometheus and Grafana

observability/prometheus.yml

```
global:
  scrape_interval: 5s

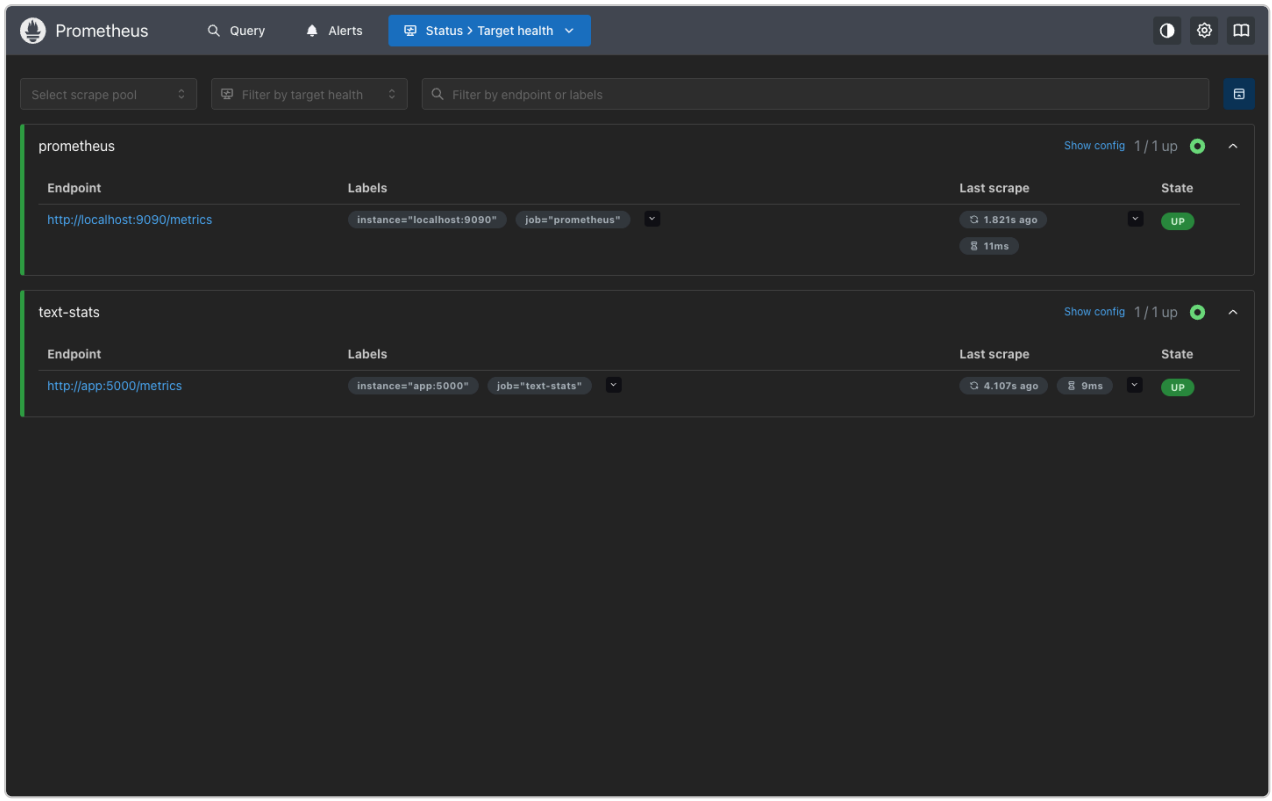
scrape_configs:
  - job_name: text-stats
    static_configs:
      - targets: ["app:5000"]
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]
```

```
$ docker compose -f observability/compose.yml up -d --build app prometheus grafana 2>&1 | grep -E 'S
tarted|Running|Healthy|Built'
 Image lab3-observability-app Built
 Container lab3-observability-prometheus-1 Started
 Container lab3-observability-grafana-1 Started
 Container lab3-observability-app-1 Started
$ docker compose -f observability/compose.yml ps --format '{{.Service}}  {{.Image}}  {{.Status}}
{{.Ports}}'
app  lab3-observability-app  Up 15 seconds (healthy)  127.0.0.1:8300->5000/tcp
grafana  mirror.gcr.io/grafana/grafana:13.3.0  Up 15 seconds  127.0.0.1:3390->3000/tcp
prometheus  quay.io/prometheus/prometheus:v3.16.0  Up 15 seconds  127.0.0.1:9390->9090/tcp
$ curl -s http://localhost:8300/healthz
{"healthy":true,"release":"3.0.0"}
```

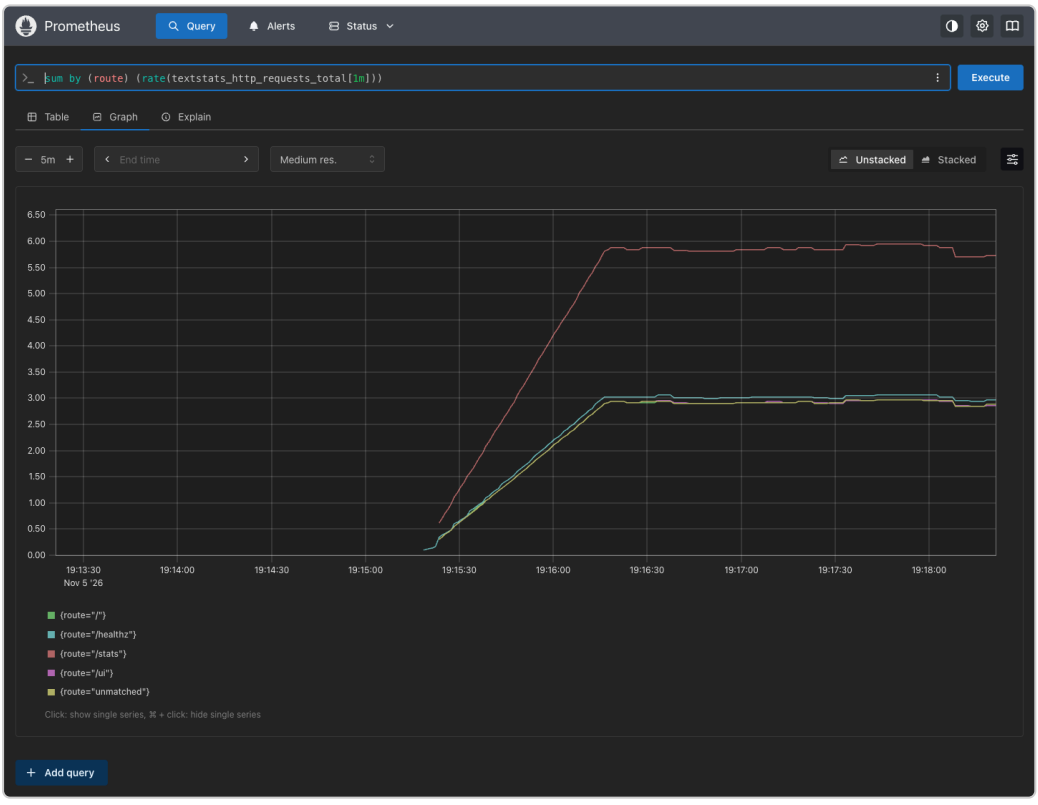
The datasource and the dashboard are provisioned from files in `observability/grafana/`, so Grafana has the dashboard on first start with no manual clicks.

Verification after 60 s of traffic

```
$ curl -s http://localhost:8300/metrics | grep -E '^textstats_(http_requests_total|words_processed_t
otal)'
textstats_http_requests_total{method="GET",route="/healthz",status="200"} 182.0
textstats_http_requests_total{method="GET",route="/",status="200"} 174.0
textstats_http_requests_total{method="GET",route="/ui",status="200"} 174.0
textstats_http_requests_total{method="POST",route="/stats",status="200"} 174.0
textstats_http_requests_total{method="POST",route="/stats",status="400"} 174.0
textstats_http_requests_total{method="GET",route="unmatched",status="404"} 174.0
textstats_words_processed_total 1392.0
$ curl -s http://localhost:9390/api/v1/targets | jq -r '.data.activeTargets[] | "\(.labels.job) \(.s
crapeUrl) \(.health)"'
prometheus http://localhost:9090/metrics up
text-stats http://app:5000/metrics up
$ curl -s --get http://localhost:9390/api/v1/query --data-urlencode 'query=sum by (route) (rate(text
stats_http_requests_total[1m]))' | jq -r '.data.result[] | "\(.metric.route) \(.value[1]) req/s"'
/healthz 3.036363636363636 req/s
/ 2.902852424242424 req/s
/ui 2.902852424242424 req/s
/stats 5.805704848484848 req/s
unmatched 2.902852424242424 req/s
$ curl -s http://localhost:3390/api/health | jq -c '{database,version}'
{"database":"ok","version":"13.3.0"}
$ curl -s http://localhost:3390/api/dashboards/uid/text-stats | jq -r '.dashboard.title, (.dashboar
d.panels[].title)'
Text Stats Service
Service up
Requests per second
Error ratio (4xx + 5xx)
Words analysed
Requests per second by route
p95 latency by route
```



Prometheus target health: both scrape targets UP



Prometheus query: request rate per route



Provisioned Grafana dashboard under live traffic

Observations

- Both targets are UP and scraped every 5 s.
- The traffic script deliberately sends 2 bad requests in every 6 (a non-string `text` and an unknown URL). The error-ratio panel correctly shows about 33% and turns red, because its thresholds are 5% (orange) and 20% (red).
- The first dashboard showed p95 latency as a flat 4.75 ms. The default histogram buckets start at 5 ms, so every sub-millisecond request landed in the first bucket. Adding buckets from 0.5 ms shows the real p95 of about 0.48 ms.
- Requests to `/metrics` are not counted, so scraping does not inflate the request rate.